SoftwareX 34 (2026) 102617



Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx





pynasonde: An open-source Python library for ionosonde data processing

S. Chakraborty [a,*], T. Bullett [b], A. Barjatya [a], J. Mabie [b]

[a] Embry-Riddle Aeronautical University, Daytona Beach, FL, USA
[b] Fourth State Communications, Cheyenne, WY, USA

ARTICLE INFO

Keywords:
Radio wave
Ionosonde
VIPIR
Space weather
Python
Ionosphere

ABSTRACT

The Earth's ionosphere, a charged and magnetized part of the upper atmosphere, is influenced by extraterrestrial drivers, such as solar wind and interplanetary magnetic fields (IMF), as well as by terrestrial processes, including acoustic–gravity waves and thunderstorms. Variability across spatial and temporal scales produces dynamic responses that affect radio propagation, space weather, and technological systems. Medium and High-frequency (MF/HF: 1–20 MHz) radio sounding provides a powerful diagnostic for sensing these conditions and monitoring ionospheric variability. We present *pynasonde*, an open-source Python application for ionospheric radio observations from MF/HF sounders, such as Digisonde and Vertical Incidence Pulsed Ionosphere Radar (VIPIR). Tailored for space weather, it integrates tools for reading, plotting, echo recognition, noise discrimination, and trace classification. By transforming raw datasets from vertical sounders into geophysical parameters, *pynasonde* v1.0.0 delivers reliable insights into ionospheric state and dynamics. This paper presents *pynasonde*'s modular, extensible design, which lowers the barrier to precision ionospheric analysis, supporting applications in space weather forecasting, radio communication, and ionospheric research, and concludes with plans for future extensions to this open source software.

Metadata

Code metadata.

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/shibaji7/pynasonde
C3	Permanent link to Reproducible Capsule	https://github.com/shibaji7/pynasonde
C4	Legal Code License	MIT
C5	Code versioning system used	Git 2.34.1
C6	Software code languages, tools, and services used	Python 3.11
C7	Compilation requirements, operating environments & dependencies	All the dependencies are listed under Documentation
C8	If available Link to developer documentation/manual	https://pynasonde.readthedocs.io/en/latest/
C9	Support email for questions	chakras4@erau.edu

* Corresponding author.
Email address: chakras4@erau.edu (S. Chakraborty).

https://doi.org/10.1016/j.softx.2026.102617
Received 6 November 2025; Received in revised form 8 March 2026; Accepted 14 March 2026
Available online 21 March 2026
2352-7110/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

1. Motivation and significance

The ionosphere is a magnetized, electrically charged region of the upper atmosphere whose dynamics are driven by both terrestrial and extraterrestrial processes [e.g., 1–3]. These drivers generate multi-scale variability that affects radio wave propagation, particularly in the high-frequency (HF; 3–30 MHz) band [e.g., 4,5], along with satellite communication and navigation systems [e.g., 6–8], and other critical infrastructures. Accurate and timely diagnostics of bottomside ionospheric variability are therefore essential for both scientific understanding [e.g., 4] and operational space weather applications.

A variety of ground-based and space-borne instruments provide remote-sensing and in situ measurements that capture the ionosphere's state, dynamics, and coupling to external drivers [2]. The growing demand for space weather forecasting from satellite operators, navigation services, and defense agencies underscores the need for integrated, multi-instrument analyses [9]. Vertical incidence sounders, such as Digisonde [e.g., 10] and VIPIR [e.g., 4,11,12], play a pivotal role in these conjunction studies. Similar to how several Python libraries exist for accessing satellite [e.g., 13] and ground-based datasets [e.g., 14], we present here an open-source Python library—*pynasonde*—designed specifically to read, plot, and analyze Digisonde and VIPIR datasets.

At the time of writing, documentation for Digisonde instruments is available through the University of Massachusetts Lowell (UML) website [15–17]. Software packages for reading, plotting, and performing preliminary analyses of Digisonde data are also distributed by UML and implemented primarily in Java [15]. These tools, however, are organized as separate programs primarily dedicated to specific data products. Documentation and software for VIPIR datasets were historically hosted on NOAA's public FTP servers [18], but portions of these resources are not widely available, as NOAA no longer supports an ionosonde program.

Motivated by both historical and ongoing studies, we have consolidated these resources into a unified, open-source Python framework, *pynasonde*, accompanied by comprehensive online documentation in the Read the Docs (**RTD**) format [19]. Recognizing that bottomside ionospheric sounders continue to play a pivotal role in space weather research, this effort aims to provide a modern, pythonic implementation to facilitate current and future investigations using Digisonde and VIPIR datasets.

Instruments covered under pynasonde library

The primary objective of this library, *pynasonde*, is to read, process, and plot the observations from ground-based near-vertical (can be programmed as oblique sounders too) incidence MF/HF sounders, including Digisondes and VIPIR. Although both instruments may seem similar at first glance, VIPIR incorporates advanced, next-generation hardware that enables significantly higher-resolution ionospheric measurements.

The Digisonde is a digital ionosonde system developed by the University of Massachusetts Lowell Center for Atmospheric Research and is widely used for vertical incidence soundings of the ionosphere [16]. It operates by transmitting radio pulses (typically between 1–20 MHz) vertically, commonly referred to as NVIS (near-vertical incidence sounding), into the ionosphere and recording the reflected signals to construct ionograms [15]. Though the user can preselect the frequency step size, a typical ionogram consists of approximately 500 frequency steps, corresponding to a 40 kHz interval between successive frequencies [10,20]. These ionograms are used to derive key ionospheric parameters [21,22], including the F2 layer critical frequency (foF2), virtual and true heights (h'F, hmF2), and electron density profiles via inversion algorithms such as POLAN [23–25].

The VIPIR (Vertical Incidence Pulsed Ionospheric Radar) is a modern software-defined radar system [11,12], offering flexible and high-fidelity measurements of the ionosphere [26]. Like the Digisonde, VIPIR transmits radio pulses vertically and records echoes from different ionospheric layers. VIPIR's design is primarily focused on maximizing dynamic range, transmitted power, and frequency resolution (typically 10–100 kHz) [26]. VIPIR data products include scaled ionograms, Doppler spectra, and electron density profiles, which are particularly valuable for capturing rapid changes in ionospheric structure during dynamic events [27]. Fig. 1 illustrates the basic principles of high-frequency (HF) propagation used by Digisonde and VIPIR instruments. These systems transmit radio pulses that reflect from ionospheric layers—the D, E, and F regions—depending on the frequency and electron density profile [28]. Near-vertical incidence soundings provide altitude-resolved information on plasma density [29], while oblique propagation paths enable the study of horizontal gradients [4], tilts, and traveling ionospheric disturbances (TIDs) [30,31].

2. Software description

The primary objective of the *pynasonde* library is to provide tools for reading, visualizing, and semi-autonomously processing the datasets, with the aim of cleaning and extracting scaled parameters from ionograms. Note that, in this version, the scaling method is implemented only for VIPIR vertical ionograms. However, the underlying implementation is based on computer vision techniques and can be readily extended to Digisonde ionograms, including both vertical and oblique soundings. Its modular architecture supports a wide range of file formats, including those produced by Digisonde instruments (`DFT`, `RSF`, `SBF`, `SAO` [versions 4.3 and 5.0], `DVL`, `SKY`, `EDP`) [10,20,32] and VIPIR systems (`RIQ`, `NGI`) [18,33]. These formats span both binary and ASCII structures, as summarized in Table 1. Detailed descriptions of the file types and internal parameters are provided on the project documentation page, i.e., in RTD.

In its current release (v1.0.0) [34], *pynasonde* includes operational features for plotting processed datasets and performing semi-autonomous routines to clean ionograms and identify their scaled

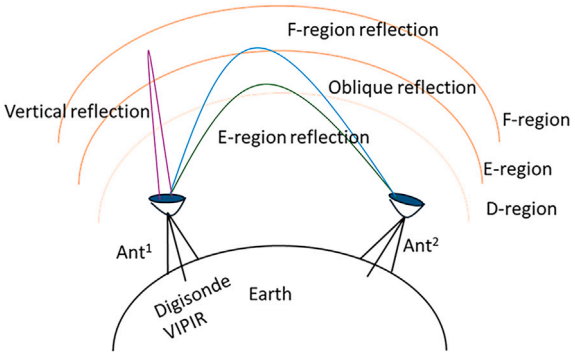


Fig. 1. Schematic diagram of Digisonde and VIPIR systems illustrating vertical and oblique high-frequency (HF) propagation through the D-, E-, and F-regions of the ionosphere. Antenna (1) represents a Digisonde or VIPIR operating in near-vertical incidence mode, while antennas (1) and (2) together depict the VIPIR configuration for oblique sounding.

Table 1
Description of primary data file formats. A details on the file formats are described in the attached supplementary document.

Extension	Type	Source Instrument	Description
RSF	Binary	Digisonde	Complete sounding information
SBF	Binary	Digisonde	Complete sounding information
SAO4.3	ASCII	Digisonde	ARTIST scaled data with echo traces
SAO5.0	XML	Digisonde	ARTIST scaled data with echo traces
DVL	ASCII	Digisonde	Derived Doppler velocity
SKY	ASCII	Digisonde	Doppler information from individual echoes
RIQ	Binary	VIPIR	Raw I/Q data
NGI/SDR	Binary	VIPIR	NetCDF processed ionograms

parameters. This functionality enables researchers to efficiently transform raw data into standardized, physically meaningful outputs, thereby lowering the barrier to reproducible and high-precision ionospheric analysis.

2.1. Software architecture

The library is organized into two primary modules: `digisonde` and `vipir`, each dedicated to the corresponding instrument introduced in the Motivation and Significance section. Within each submodule, specialized parsers handle the supported data file types, accompanied by plotting tools for visualization. Fig. 2 illustrates the workflow for both modules, including data parsing (reading), plotting, and processing (for `vipir` module). The workflow figure illustrates the primary methods (right-hand side) and the currently supported file and data types (left-hand side) implemented in this version of *pynasonde* under the `digisonde` and `vipir` modules. The file types handled by the `digisonde` and `vipir` modules are color-coded according to their associated plotting styles. Within the `digisonde` module, the `RSF` and `SBF` file types contain information related to vertical ionograms and are visualized using the `ionogram()` plotting method. The `SAO`, `SKY`, and `EDP` file types store derived parameters and outputs produced by the ARTIST 5.0 algorithm. Among these datasets, `SAO` and `EDP` contain ionospheric plasma parameters in time-series format and are visualized using the `stackplot()` method to facilitate comparative analyses. The `skymap()` plotting routine is used for `SKY` datasets and visualizes echo locations for individual soundings as instrument fields of view in polar coordinates. Finally, the `DFT` datasets contain Doppler-resolved spectral information of received echoes as a function of sounding frequency and range gate. These data are visualized using the `stackplot()` method to enable comparative analysis of Doppler spectra across different range gates. Within the `vipir` module, the `SDR` files contain calibrated and geolocated data products derived from the raw `RIQ` files, which store the original in-phase and quadrature (I/Q) channel measurements. The `ionogram()` and `fti()` plotting routines are used to visualize observational products derived from the `SDR` files, including frequency–height ionograms and frequency-binned time–intensity representations.

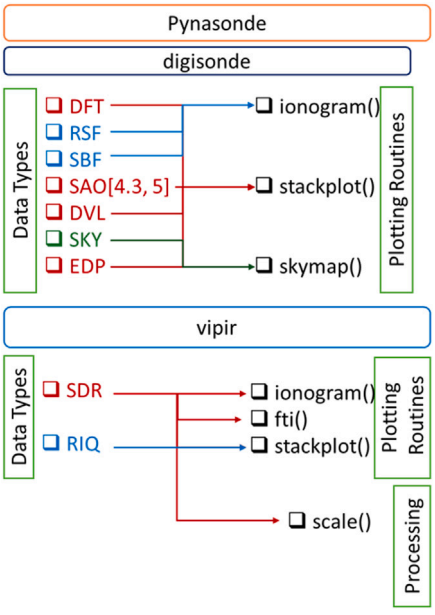


Fig. 2. Schematic diagram of the two modules, `digisonde` and `vipir`, implemented in *pynasonde*. Data types are color-coded according to the plotting methods used to generate their respective dedicated plots. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article).

The `stackplot()` method can be applied directly to raw I/Q voltage datasets for instrument-level diagnostics and signal characterization. Additionally, the `scale()` method provides automated ionogram scaling capabilities by extracting geophysical parameters from the `SDR`-based ionogram products.

2.2. Software functionalities

The primary function of this version of *pynasonde* is to read and visualize the Digisonde and VIPIR outputs. For visualization, *pynasonde* provides both generic plotting functions, applicable across datasets, and instrument-specific routines for generating dedicated plots. In addition, the `vipir` module includes a processing unit with one key method: `scale()`, which extracts scaled ionospheric parameters using simple image segmentation techniques and derives echo traces from raw I/Q datasets.

Section 3 provides illustrative examples and code snippets demonstrating the operation of these functions.

2.2.1. Working principle of `scale()` method

The `scale()` method is a Machine Learning-based (ML) tool within the *pynasonde* library designed for the automatic scaling of ionograms. Leveraging sophisticated image segmentation and clustering techniques, this tool reduces the need for manual intervention, delivering reliable and accurate extraction of ionospheric parameters. The tool employs methods like fuzzy K-means clustering, Otsu's thresholding, and DBSCAN (Density-Based Spatial Clustering of Applications with Noise), allowing researchers to process ionogram data more efficiently, especially during complex space weather events.

The `scale()` workflow begins with raw ionogram data, which is progressively refined to extract key ionospheric parameters. The raw ionogram (Fig. 3a) contains noisy echoes that are first filtered using a frequency–noise profiler and virtual-height constraints (Fig. 3b). The filtered data are then segmented with fuzzy K-means [35] (Fig. 3c), capturing gradual transitions between ionospheric structures. Otsu's thresholding [36] converts the segmented [37] output to a binary image (Fig. 3d), isolating true echoes from residual noise. Next, DBSCAN [38] identifies coherent echo traces by grouping dense clusters and discarding outliers (Fig. 3e). The final stage (Fig. 3f) estimates physical parameters such as the critical frequency (fo_s) and maximum (virtual) height (h'_s), enabling robust characterization of ionospheric behavior during dynamic events, including the 2024 Great American Eclipse.

2.2.2. Working principle of `ionogram()`, `stackplot()`, `skymap()`, and `fti()` methods

These plotting methods are designed to visualize specific classes of ionospheric data products for analysis. The `ionogram()` method is used to display ionograms recorded by MF/HF sounders: within the `pynasonde.digisonde` module it supports `RSF` and `SBF` products, while the corresponding method in the `pynasonde.vipir` module supports `SDR` (Sensor Data Records or `NGI`) datasets. This functionality enables comparative visualization of ionograms from multiple instruments at the same time to assess spatial variability, as well as from a single instrument across different times to examine temporal variability.

The `stackplot()` method is designed to visualize data products that are predominantly time-series in nature. This includes the `DVL`, `SAO`, and `EDP` datasets, which store derived ionospheric parameters as functions of time. The primary objective of this method is to enable comparative visualization of time-series data from multiple instruments to assess spatiotemporal variability. In addition, the `stackplot()` method supports visualization of Doppler spectra contained in `DFT` data products, allowing users to compare spectral characteristics across different range gates and time intervals.

The `skymap()` method is specifically designed to visualize `SKY` data products in polar coordinates. It facilitates comparison of Doppler information from returned echoes as a function of location within the two-dimensional field of view of the instrument. For vertical sounders,

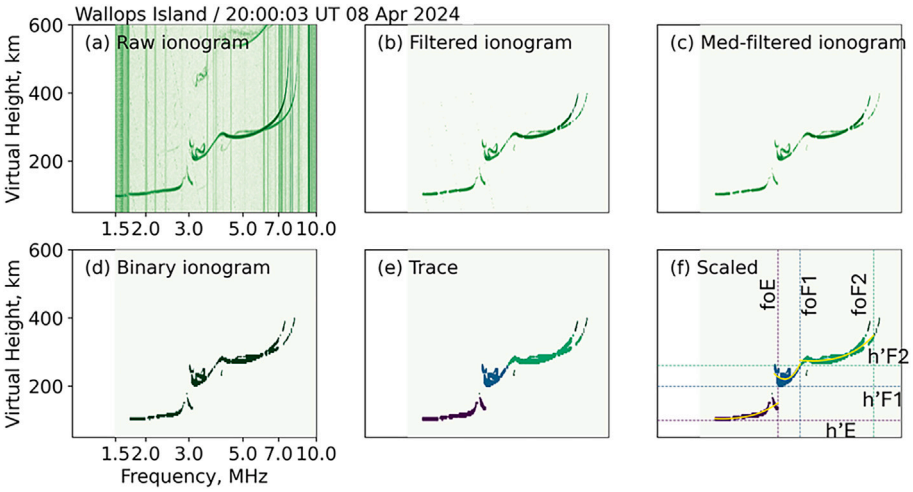


Fig. 3. Example showing illustration of the various stages of `scale()` in action. It presents the output from different stages of the tool applied to a vertical ionogram from Wallops Island VIPIR (WI937). The event occurred at 15:50:03 UT on April 4, 2024, and was related to the 2024 Great American Eclipse (GAE). The stages shown include: (a) raw vertical ionogram, (b) filtered ionogram (based on a frequency-noise profiler and expected virtual height), (c) segmented ionogram using fuzzy K-means clustering, (d) Binary image created from the segmented part using Otsu's thresholding, (e) traces identified using density-based clustering (DBSCAN), (f) Estimated parameters fo_s (critical frequency) and h'_s (layer virtual height).

this field of view is defined by azimuth angles (0°–360°) and zenith angles (typically 0°–40°).

Finally, the `fti()` method is used to generate frequency-binned time–intensity plots from VIPIR data. This visualization enables comparison of returned signal strength (echo power) across different frequency bands, which correspond to different range cells or virtual heights. The frequency binning is user-defined and can be as fine as the native frequency resolution of the dataset determined during instrument operation. This method allows users to examine the evolution of ionospheric features as a function of effective reflection height.

3. Illustrative examples

The *pynasonde* library follows recommended community best practices [39]. It is implemented in Python (version 3.11 [40] or later) and makes use of both standard and widely adopted community packages, which are summarized in Table 2. At present, there are no strict version requirements for these dependencies, and any missing modules will be installed automatically when the package is installed via the Python Package Index (PyPI).

The preferred installation method is through `pip`: `pip install pynasonde`.

Alternatively, the source code can be obtained directly from the GitHub repository and installed locally. This option is particularly useful for users who wish to modify or extend the library for specialized use cases. The command-line instructions provided in Listing 1 demonstrate how to install *pynasonde* in development mode, ensuring that any changes made to the source code are immediately reflected when the package is imported.

Table 2
Version v1.0.0 library dependencies.

Required	Testing
numpy [41]	pytest [42]
pandas [43]	coverage[toml] [44]
matplotlib [45]	pytest-cov [46]
opencv-python-headless [37]	
SciencePlots [47]	
timezonefinder [48]	
scipy [49]	
loguru [50]	

Following illustrative examples presented in this section are designed to demonstrate consistent, end-to-end usage patterns of the *pynasonde* library. For each data product, the examples explicitly describe the workflow from data import and parsing, through optional processing or filtering steps, to final visualization and figure generation. The focus is on exposing the user-facing API and operational sequence required to reproduce each analysis. While the main text provides a step-by-step conceptual description of these workflows, complete executable scripts corresponding to each example are provided in the GitHub repository to support full implementation and reproducibility.

3.1. Reading and plotting `SAO` files

The example script `sao.py`, provided in the `examples/digisonde/` directory of the GitHub repository, demonstrates two complementary workflows for analyzing Digisonde `.SAO` data using the *pynasonde* library. In the first workflow, users load one or more directories of processed `.SAO` files using the `SaoExtractor.load_SAO_files()` method with the `height_profile` option. This step parses scaled ionogram products and derives electron density profiles as a function of time and real height (or altitude). The resulting time–height electron density distribution is visualized using the `SaoSummaryPlots` class, producing stack plots that highlight the temporal evolution of plasma density.

In the second workflow, the same `.SAO` files are reloaded using the `scaled` option to extract summary ionospheric parameters, including the F2-layer critical frequency (`foF2`) and peak height (`hmF2`). These parameters are visualized as time series on dual y-axes using the `plot_TS()` method, enabling direct comparison of layer frequency and height variations. Axis limits, time windows, and legends are customized to emphasize a single-day interval.

Fig. 4 illustrates the results of these workflows for Digisonde observations acquired during the 14 October 2023 eclipse event, showing (a) eclipse-driven changes in derived plasma density and (b) the corresponding evolution of `foF2` and `hmF2`. Together, these examples demonstrate

```
git clone https://github.com/shibaji7/pynasonde/
cd pynasonde
pip install -e .
```

Listing 1. Download and install repository from GitHub in development mode.

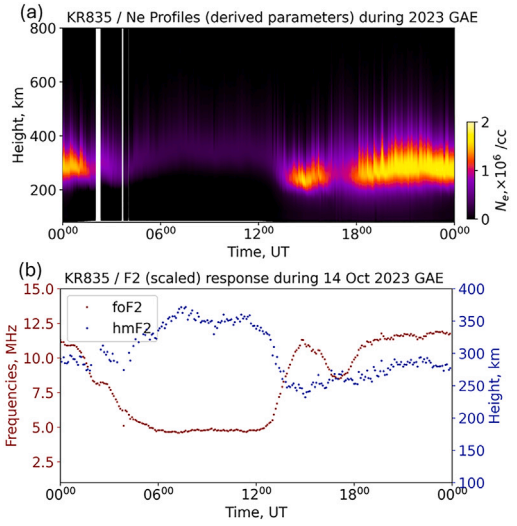


Fig. 4. (a) Electron-density height profiles for the Kirtland station during the 14 October 2023 Great American Annular Eclipse. (b) Dual-axis depiction of foF_2 and hmF_2 from the same day, illustrating how the F_2-layer critical frequency and peak height varied throughout the eclipse.

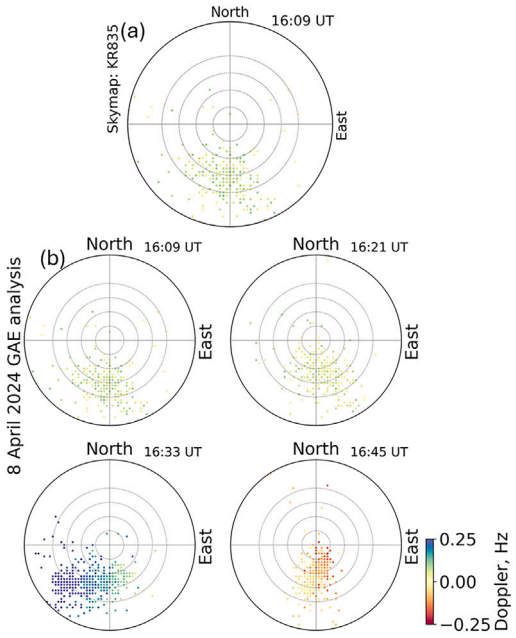


Fig. 5. (a) Single-map output highlighting Doppler frequency shifts observed at Kirtland during the 08 April 2024 Great American Eclipse (16:09 UT). (b) multi-panel layout from the same station eclipse day, ideal for contrasting multiple time intervals or stations in a single figure.

the complete analysis sequence from data import and parameter extraction to publication-ready visualization, and serve as a practical template for applying the same methodology to other campaigns or stations.

3.2. Reading and plotting *SKY* maps

The example script `sky.py`, provided in the `examples/digisonde/` directory of the GitHub repository, demonstrates end-to-end workflows for visualizing Digisonde `.SKY` data using the *pynasonde* library. In this example, users first instantiate a `SkyExtractor` object by providing the path to a processed `.SKY` file along with optional flags to load auxiliary frequency and angle tables. The `extract()` method is then invoked to parse the raw skymap data and associated metadata, after which the parsed structure is converted into a tabular format using `to_pandas()` for downstream analysis and visualization.

For single-sounding analysis, the resulting dataset is passed to the `SkySummaryPlots` class, and the `plot_skymap()` method is used to generate a polar skymap in which Doppler frequency is visualized as a function of azimuth and zenith angle. This representation highlights the spatial distribution and Doppler characteristics of ionospheric echoes within the two-dimensional field of view of the instrument while discarding range information.

The same workflow is extended to multi-panel visualization by iterating over a sequence of `.SKY` files acquired at different times. In this case, multiple skymaps are rendered within a single figure using a grid layout, enabling direct comparison of echo locations and Doppler signatures across successive soundings. Time annotations and consistent color scales are applied to facilitate interpretation of temporal evolution.

Fig. 5(a) presents an isolated skymap illustrating the Doppler characteristics of ionospheric echoes, while Fig. 5(b) shows a sequence of four skymap panels depicting the temporal evolution of ionospheric reflection points during the 8 April 2024 Great American Eclipse. Together, these examples demonstrate the complete analysis sequence from data import and parsing to single- and multi-panel visualization, and illustrate how skymap products can be used to investigate ionospheric tilts, wave propagation effects, and spatial dynamics of reflection regions.

3.3. Reading and plotting *DVL* files

The example script `dvl.py`, provided in the `examples/digisonde/` directory of the GitHub repository, demonstrates a complete end-to-end workflow for visualizing Digisonde `.DVL` data using *pynasonde*. In this example, users first collect one or more directories containing processed `.DVL` files and load them using the `DvlExtractor.load_DVL_files()` static method, which parses individual files (typically corresponding to single soundings) and aggregates them into a continuous time-series dataset.

The resulting data structure is then passed to `plot_dvl_drift_velocities()`, a class method, which generates stacked panels of horizontal and vertical bulk Doppler velocities. The plotting routine allows further customization of axis limits, time windows, and overlays, such as the addition of mean virtual height using a secondary axis. The final figure is exported in publication-ready format for inclusion in documentation or scientific manuscripts.

Fig. 6 illustrates this workflow for Digisonde observations acquired at Kirtland, NM during the 14 October 2023 eclipse event, highlighting eclipse-driven ionospheric velocity perturbations. This example serves as a practical template for reproducing the full analysis sequence, from data import to final visualization.

3.4. Reading and plotting *RIQ* datasets

The example script `proc_riq.py`, provided in the `examples/vipir/` directory of the GitHub repository, demonstrates an end-to-end workflow for reading and visualizing raw VIPIR `.RIQ` datasets using the *pynasonde* library. In this example, users first create a `RiqDataset` object by loading a processed `.RIQ` file through the `RiqDataset.create_from_file()` static method, which applies the appropriate VIPIR configuration and character encoding. This step parses the raw I/Q measurements and associated metadata required for ionogram construction.

The parsed dataset is then converted into an ionogram representation using the `get_ionogram()` method. To enhance echo visibility and suppress background noise, an adaptive gain procedure is applied using the `adaptive_gain_filter()`, which performs baseline noise removal and optional median filtering. The resulting cleaned ionogram is subsequently passed to the `Ionogram` plotting class, where returned signal power is visualized as a function of frequency and virtual height.

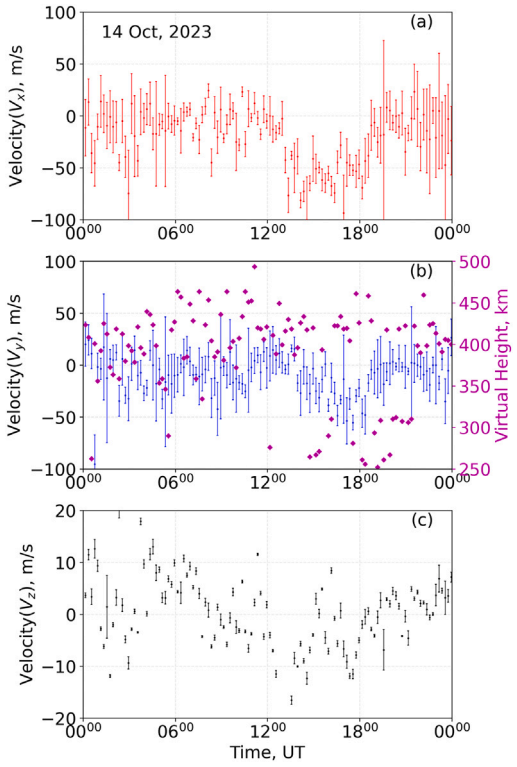


Fig. 6. Drift velocities along (a) north-south, (b) east-west, and (c) zenith, the 14 October 2023 Great American Annular Eclipse, highlighting plasma dynamics throughout the day. Virtual heights of derived velocity observations are shown in the right axis of panel (b).

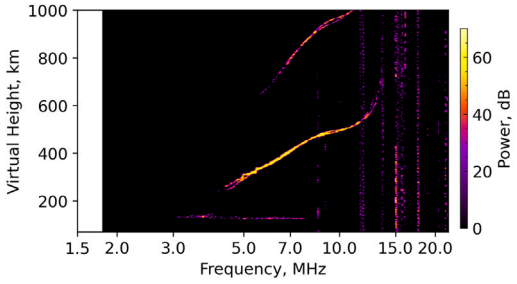


Fig. 7. Ionogram derived from `PL407_2024058061501.RIQ` file, filtered with the adaptive gain procedure to suppress baseline noise.

Fig. 7 shows an example ionogram derived from the `PL407_2024058061501.RIQ` file, illustrating returned power as a function of virtual range and frequency after adaptive gain filtering. This example demonstrates the complete analysis sequence from raw VIPIR I/Q data ingestion to noise-suppressed, publication-ready ionogram visualization. The corresponding `.RIQ` dataset used in this example is provided in the `examples/data/` directory to facilitate reproducibility.

3.5. Reading and plotting SDR files using `fti()` method

The example module `scale_module.py`, provided in the `examples/vipir/` directory of the GitHub repository, demonstrates an end-to-end workflow for processing and visualizing VIPIR `SDR` ionogram products using the *pynasonde* library. In this example, users first stage a selected time window of `SDR` files and load the corresponding ionogram datasets using the `DataSource` interface, which aggregates individual ionograms into analysis-ready data cubes.

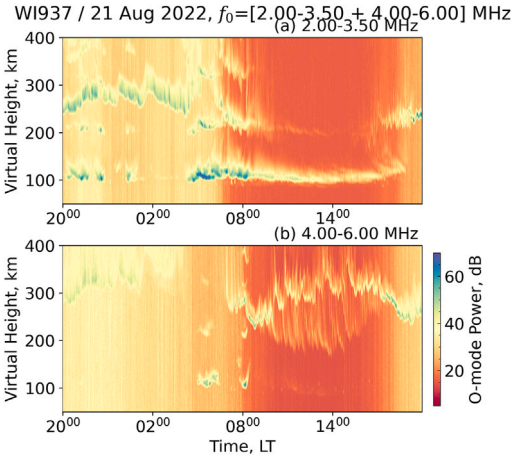


Fig. 8. FTI stack plots from station WI937 on August 22, 2022, displays O-mode power dynamics across a full day within the specific frequency range of (a) 2–3.5 MHz and (b) 4–6 MHz.

The loaded datasets are then processed using the `AutoScaler` framework, which applies a sequence of noise characterization, filtering, and image-based segmentation steps. These include construction of a noise profile, median filtering, image segmentation, and binary trace extraction using thresholding and clustering methods. Configuration parameters controlling frequency and height ranges, segmentation thresholds, and clustering behavior are read from a user-editable configuration file, allowing the workflow to be adapted to different campaigns or operating modes.

The processed ionograms are subsequently visualized using frequency-binned time–intensity (FTI) plotting routines, which display returned signal power as a function of time and effective reflection height within user-defined frequency bands. Fig. 8 shows an example application of this workflow for VIPIR observations from the Wallops Island station during a sporadic E (*Es*) event on 21 August 2022. The two panels correspond to frequency bands of 2–3.5 MHz and 4–6 MHz, respectively, revealing a persistent *Es* layer near 100 km virtual height throughout the day and patchy nighttime structures. Between 08 and 12 UT, the *Es* layer intensifies and extends to nearly 6 MHz.

The VIPIR system operates using chirp-frequency soundings rather than broadband excitation. The `SDR` data products therefore contain returned signal power as a function of discrete transmitted frequency and virtual height for each sounding. In Fig. 8, the displayed panels do not represent a single broadband transmission; instead, they correspond to user-defined frequency bins (e.g., 2–3.5 MHz and 4–6 MHz) applied during post-processing. For each selected frequency interval, the returned power is aggregated and visualized as a function of time, producing a frequency-binned time–intensity (FTI) representation. This approach enables examination of ionospheric structures within specific frequency bands and highlights their temporal evolution.

This example illustrates the complete analysis sequence from data ingestion and automated scaling to frequency-resolved visualization, and highlights the capability of *pynasonde* to investigate fine-scale ionospheric variability using high-resolution VIPIR observations. Due to their size, the full `SDR` datasets used in this example are not publicly archived but are available from the authors upon reasonable request.

4. Impact

This library modernizes and unifies existing tools for reading, plotting, and analyzing observations from vertical incidence MF/HF instruments. By providing a Python-based framework with a modular design, *pynasonde* enables next-generation researchers and data analysts to efficiently perform multi-point [e.g., 51,52] and multi-instrument

Table 3
Expected functionalities associated with tentative future releases.

Tentative future versions	Expected new functionalities
V2.0.0	2D raytracing, modified POLAN, and oblique ionograms.
V3.0.0	Reading the VIPIR datasets and incorporating directional and polarization information.
V4.0.0	3D raytracing and Cisium plotting capabilities.

studies [e.g., 7,9,53], integrating observations from both ground-based and space-borne platforms [e.g., 54]. The framework lowers the technical barrier to data access and promotes reproducible analysis workflows that are easily adaptable to different experimental setups and instruments [39].

A recently published article [55] uses *pynasonde* to analyze ionospheric G-condition observed during the 2017 Great American Eclipse (GAE). We have also begun applying the library's modules to analyze observations from Digisondes located at Kirtland, Sulf Site, and Wallops Island during the 2023 and 2024 Great American Eclipses [56]. These case studies demonstrate the capability of *pynasonde* to handle large, event-driven datasets and to extract key ionospheric parameters for studying eclipse-driven variability. Additionally, the `vipir` module has been employed for complementary analyses, including the investigation of sporadic-E layer dynamics [57] and coordination with rocket-based measurements under the SpEED Demon campaign [58]. A dedicated scientific manuscript presenting these results is currently in preparation.

5. Future development & conclusion

This version (v1.0) of *pynasonde* primarily focuses on reading, plotting, and basic trace identification and processing. Future development will expand the library's functionality to include support for additional file types, implementation of modified POLAN inversion algorithms, and building *python*-based two- and three-dimensional ray-tracing capabilities. The future version will be able to read and identify traces on the oblique ionograms. We plan to incorporate algorithms to detect and infer the directional and polarization information in the VIPIR datasets. We also plan to extend the framework to accommodate raw Digisonde datasets and to enable more advanced analysis routines built upon the current version.

A tentative roadmap for upcoming releases is outlined in Table 3. We invite the broader research community to use, validate, and contribute to the library's continued development. Although not exhaustive, the outlined roadmap provides a representative pathway for the next several major versions of *pynasonde*.

The *pynasonde* library provides a flexible and Pythonic framework that enables HF and ionospheric (geospace) researchers to access both raw and processed datasets from Digisonde and VIPIR instruments, generate quick diagnostic plots, and produce publication-quality visualizations. The library also facilitates access to scaled parameters and incorporates auto-scaling routines based on image segmentation methods.

By creating ease of access to the code with examples, *pynasonde* supports analyses ranging from individual instrument studies to multi-point, event-based, and statistical investigations. The included example codes can be easily adapted to specific research needs, allowing users to streamline workflows from data access through preliminary processing. By simplifying the tasks of reading, plotting, cleaning, and analyzing vertical sounder datasets, the software enables efficient sanity checks, robust data interpretation, and coordinated multi-instrument studies for advancing ionospheric research.

Future developments will focus on extending *pynasonde* with built-in ray-tracing and homing-in modules to conduct conjunction studies and improve the interpretation of recorded ionograms. Additional efforts will include integration with advanced inversion tools, such as next-generation POLAN modules, and the expansion of automated workflows to better support large-scale space weather monitoring and forecasting.

CRediT authorship contribution statement

S. Chakraborty: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Conceptualization. **T. Bullett:** Writing – review & editing, Software, Resources, Data curation. **A. Barjatya:** Writing – review & editing, Supervision, Resources, Project administration, Methodology. **J. Mabie:** Writing – review & editing, Software, Methodology, Data curation.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this article, the author(s) used ChatGPT to check grammar and the English language. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Funding and support for this work were provided under NASA Grants 80NSSC19K0773 and the Center for Space and Atmospheric Research at ERAU. SC also thanks the National Science Foundation (NSF) for support under Grants AGS-2512183 and AGS-2452540.

Appendix A. Supplementary data

Supplementary data for this article can be found online at doi:10.1016/j.softx.2026.102617.

References

[1] Davies K. Ionospheric radio. Electromagnetics and Radar Series. Institution of Engineering & Technology; 1990. ISBN 9780863411861. https://books.google.com/books?id=qdWUKSj5PCcC.
[2] Schunk RW, Nagy AF. Ionospheres. 2nd ed. Cambridge Atmospheric and Space Science Series; 2004.
[3] Chakraborty S, Baker JBH, Ruohoniemi JM, Kunduri B, Nishitani N, Shepherd SG. A study of superdarn response to co-occurring space weather phenomena. Space Weather 2019;17(9):1351–63. https://doi.org/10.1029/2019SW002179. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2019SW002179.
[4] Bullett T, Mabie J. Vertical and oblique ionosphere sounding during the 21 August 2017 solar eclipse. Geophys Res Lett 2018;45(8):3690–7. https://doi.org/10.1002/2018GL077413. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2018GL077413.
[5] Fiori RAD, Koustov AV, Chakraborty S, Ruohoniemi JM, Danskin DW, Boteler DH, Shepherd SG. Examining the potential of the super dual auroral radar network for monitoring the space weather impact of solar x-ray flares. Space Weather 2018;16(9):1348–62. https://doi.org/10.1029/2018SW001905. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2018SW001905.
[6] Ishii M, Berdermann J, Forte B, Hapgood M, Bisi MM, Romano V. Space weather impact on radio communication and navigation. Adv Space Res 2024. https://doi.org/10.1016/j.asr.2024.01.043. ISSN 0273-1177. https://www.sciencedirect.com/science/article/pii/S0273117724000863.
[7] de Abreu AJ, Correia E, Venkatesh K, Pignalberi A, Pezzopane M, Pillat VG, Fagundes PR, Gende M, de Jesus R. Effects of x2.8-class solar flare on the ionosphere occurred during the recovery phase of a geomagnetic storm over south American and antarctic sectors. Adv Space Res 2025;76(8):4539–59. https://doi.org/10.1016/j.asr.2025.07.045. ISSN 0273-1177. https://www.sciencedirect.com/science/article/pii/S027311772500780X.
[8] Etchells T, Aplin KL, Berthoud L, Kalavana A, Larkins A. Extreme space weather impacts on GNSS timing signals for electricity grid management. Space Weather 2024;22(10):e2023SW003770. https://doi.org/10.1029/2023SW003770. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2023SW003770. e2023SW003770 2023SW003770.
[9] Amaechi PO, Erickson PJ, Coster AJ, Rideout W, Goncharenko L, Zhang S-R. Multi-instrument investigation of the impact of the space weather events of 6–10 September 2017. Space Weather 2021;19(9):e2021SW002806. https://doi.org/10.1029/2021SW002806. https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2021SW002806.

[10] Reinisch BW, Galkin IA, Paznukhov VV, Scali JL, Haines DM. New digisonde for research and monitoring applications. Radio Sci 2009;44(RS0A24). https://doi.org/10.1029/2008RS004115

[11] Kim E, Jee G, Ham Y-B, Zabotin N, Lee C, Kwon H-J, Hong J, Kim J-H, Bullett T. Assessment of polar ionospheric observations by Vipir/Dynasonde at Jang Bogo Station, Antarctica: part 1—ionospheric densities. Remote Sens 2022;14(12):2785. https://doi.org/10.3390/rs14122785

[12] Teraoka S, et al. Calibration of $h'es$ from vipir2 ionosondes in Japan. Earth Planets Space 2025;77(1):138. https://doi.org/10.1186/s40623-025-02138-4

[13] Grimes EW, Harter B, Hatzigeorgiu N, Drozdov A, Lewis JW, Angelopoulos V, Cao X, Chu X, Hori T, Matsuda S, Jun C-W, Nakamura S, Kitahara M, Segawa T, Miyoshi Y, Le Contel O. The space Physics environment data analysis system in Python. Frontiers in Astronomy and Space Sciences 2022;9. https://doi.org/10.3389/fspas.2022.1020815. ISSN 2296-987X. https://www.frontiersin.org/journals/astronomy-and-space-sciences/articles/10.3389/fspas.2022.1020815.

[14] Shi X, Schmidt M, Martin CJ, Billett DD, Bland E, Tholley FH, Frissell NA, Khanal K, Coyle S, Chakraborty S, Detwiller M, Kunduri B, McWilliams K. Pydarn: a Python software for visualizing superdarn radar data. Frontiers in Astronomy and Space Sciences 2022;9. https://doi.org/10.3389/fspas.2022.1022690. ISSN 2296-987X. https://www.frontiersin.org/journals/astronomy-and-space-sciences/articles/10.3389/fspas.2022.1022690.

[15] Galkin IA, Reinisch BW, Huang XQ, Khmyrov GM. Confidence score of artist-5 ionogram autoscaling. INAG Technical Memorandum University of Massachusetts Lowell 2013. https://www.ursi.org/files/CommissionWebsites/INAG/web-73/confidence_score.pdf. Quality metric and confidence evaluation for autoscaled ionograms.

[16] Galkin IA, et al. The new artist 5 for all digisondes. In: Proceedings of URSI INAG; 2008. https://www.ursi.org/files/CommissionWebsites/INAG/web-69/2008/artist5-inag.pdf. Describes improvements, autoscaling, and the SAO.XML data format for Digisonde.

[17] Reinisch BW, Galkin IA. Global ionospheric Radio Observatory (Giro); 2025. https://giro.uml.edu/[Accessed 9 October 2025]. Managed by the University of Massachusetts Lowell Center for Atmospheric Research (UMLCAR). Provides global Digisonde data, ionogram archives, and autoscaled ionospheric parameters.

[18] NOAA National Geophysical Data Center. Readriq: documentation for the Vipir Riq data format. Technical report, U.S. Department of Commerce, National Oceanic and Atmospheric Administration (NOAA), National Geophysical Data Center (NGDC); 2008. https://data.ngdc.noaa.gov/instruments/remote-sensing/active/profilers-sounders/ionosonde/documentation/VIPIR/readriq-docs.pdf. Technical documentation describing the structure, variables, and readout procedure for VIPIR RIQ files.

[19] Chakraborty S. Pynasonde: open-source Python library for precision ionospheric radio sounding; 2025. Online documentation. Available at: https://pynasonde.readthedocs.io/en/latest/.

[20] Reinisch BW, Galkin IA, Cheney GP, Kozlov AV, Huang X, Paznukhov VV. Digisonde 4d: user manual. Technical report, Lowell Digisonde International, University of Massachusetts Lowell; 2019. https://digisonde.com/pdf/Digisonde4DManual_LDI-web.pdf. Technical manual describing operation, data formats, and analysis procedures for Digisonde 4D systems.

[21] Jiang C, Yang G, Zhou Y, Zhu P, Lan T, Zhao Z, Zhang Y. Software for scaling and analysis of vertical incidence ionograms-ionoscaler. Adv Space Res 2017;59(4):968–79. https://doi.org/10.1016/j.asr.2016.11.019. ISSN 0273-1177. https://www.sciencedirect.com/science/article/pii/S0273117716306457.

[22] Shaver DJ, et al. E-region and f-region electron densities with ionosondes and firi. Remote Sens 2023;15(18):4363. https://doi.org/10.3390/rs15184363

[23] Titheridge JE. Ionogram analysis with the generalized program Polan (uag-93). UAG Technical Report 1993. Classic reference for POLAN inversion.

[24] Amarante GM. Ionogram inversion f1-layer treatment effect in ray tracing. Ann Geophys 2005;23(6):2053–62. https://doi.org/10.5194/angeo-23-2053-2005

[25] Scotto C. Estimating the vertical electron density profile from an ionogram inversion. Radio Sci 2012;47(RS0L15). https://doi.org/10.1029/2011RS004833

[26] Grubb RN. A new hf radar for the study of the ionosphere—vipir: vertical incident pulsed ionospheric radar. In: Proceedings of the International Union of Radio Science (URSI) General Assembly. U.S. Department of Commerce, NOAA National Geophysical Data Center; 2008. https://data.ngdc.noaa.gov/instruments/remote-sensing/active/profilers-sounders/ionosonde/documentation/VIPIR/Grubb-2008-New_HF_Radar-VIPIR-URSI.pdf. URSI General Assembly paper describing the design and performance of the VIPIR ionosonde system.

[27] Zabotin NA, Wright JW. Dynamics of the ionosphere as a function of the geomagnetic condition. In: Proceedings of the URSI International Symposium on radio science. Ottawa, Canada: URSI; 2006. Describes ionospheric dynamics and their dependence on geomagnetic activity.

[28] Zabotin NA, Wright JW, Zhbankov GA. Nextyz: three-dimensional electron density inversion for dynasonde ionograms. Radio Sci 2006;41(RS6S32). https://doi.org/10.1029/2005RS003352. https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2005RS003352.

[29] Kwon H-J, Jee G, Ham Y-B, Zabotin N, Lee C, Kim E, Bullett TW. Assessment of polar ionospheric observations by Vipir/Dynasonde at Jang Bogo Station, Antarctica: 2. Ionospheric ION drift velocity. J Geophys Res Space Phys 2024;129(5):e2023JA032402. https://doi.org/10.1029/2023JA032402. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2023JA032402. e2023JA032402 2023JA032402.

[30] Mabie J, Bullett T. Multiple cusp signatures in ionograms associated with rocket-induced infrasonic waves. Atmosphere 2022;13(6). https://doi.org/10.3390/atmos13060958. ISSN 2073-4433. https://www.mdpi.com/2073-4433/13/6/958.

[31] Reyes PM, Kudeki E, Lehmacher GA, Chau JL, Milla MA. Vipir and 50 mhz radar studies of gravity wave signatures in 150-km echoes observed at Jicamarca. J Geophys Res Space Phys 2020;125(8):e2019JA027535. https://doi.org/10.1029/2019JA027535. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2019JA027535.

[32] Reinisch BW, Huang X, Scali JL, Paznukhov V, Sales GS. Digisonde Network and the World Ionosphere and Thermosphere Study (Wits). In: World ionosphere and thermosphere study: handbook, chapter 13. Boulder, CO, USA: U.S. Department of Commerce, NOAA National Geophysical Data Center; 1996. https://www.ngdc.noaa.gov/stp/space-weather/online-publications/miscellaneous/world_ionosphere_thermosphere_study/Chapter13.pdf. Comprehensive overview of the Digisonde network and its contribution to the World Ionosphere and Thermosphere Study (WITS).

[33] NOAA National Geophysical Data Center. Riq specification version 1.2: data format definition for the vipir ionosonde. Technical report, U.S. Department of Commerce, National Oceanic and Atmospheric Administration (NOAA), National Geophysical Data Center (NGDC); 2009. https://data.ngdc.noaa.gov/instruments/remote-sensing/active/profilers-sounders/ionosonde/documentation/VIPIR/RIQ_Specification_v1.2.PDF. Technical specification detailing the binary data structure, variable definitions, and metadata fields for VIPIR RIQ ionosonde files.

[34] Chakraborty S. Pynasonde: open-source Python library for precision ionospheric radio sounding. Oct 2025. https://doi.org/10.5281/zenodo.17476077

[35] Bezdek JC. Pattern recognition with fuzzy objective function algorithms. Plenum Press 1981.

[36] Otsu N. A threshold selection method from gray-level histograms. IEEE Trans Syst Man Cybern 1979;9(1):62–6. https://doi.org/10.1109/TSMC.1979.4310076

[37] Bradski G. The OpenCV library. Dr Dobbs J Softw Tools 2000;120:122–5. Open Source Computer Vision Library.

[38] Xu R, Wunsch DC. Clustering algorithms in r: a survey. IEEE Trans Syst Man Cybern 2009;39(2):259–71. https://doi.org/10.1109/TSMCB.2008.2002240

[39] Burrell AG, Halford A, Klenzing J, Stoneback RA, Morley SK, Annex AM, Laundal KM, Kellerman AC, Stansby D, Ma J. Snakes on a spaceship—an overview of Python in heliophysics. J Geophys Res Space Phys 2018;123(12):10384–402. https://doi.org/10.1029/2018JA025877. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2018JA025877.

[40] Python Software Foundation. Python 3.11: a high-level programming language for general-purpose programming; 2023. Version 3.11, https://www.python.org/ [accessed 9 October 2025].

[41] Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, Berg S, Smith NJ, Kern R, Picus M, Hoyer S, van Kerkwijk MH, Brett M, Haldane A, del Río JF, Wiebe M, Peterson P, Gérard-Marchant P, Sheppard K, Reddy T, Weckesser W, Abbasi H, Gohlke C, Oliphant TE. Array programming with NumPy. Nature 2020;585:357–62. https://doi.org/10.1038/s41586-020-2649-2

[42] Krekel H, et al. Pytest: simple powerful testing with Python; 2004. Version 8.x. Open source testing framework for Python. https://docs.pytest.org/en/latest/ [Accessed 9 October 2025].

[43] McKinney W. Data structures for statistical computing in Python. 2010. https://doi.org/10.25080/Majora-92bf1922-00a. pp. 51–6.

[44] Batchelder N. Coverage.py: code coverage measurement for Python programs; 2024. Includes `coverage[toml]` configuration support. https://coverage.readthedocs.io/ [Accessed 9 October 2025].

[45] Hunter JD. Matplotlib: a 2d graphics environment. Comput Sci Eng 2007;9(3):90–5. https://doi.org/10.1109/MCSE.2007.55

[46] The Pytest Development Team. Pytest-cov: pytest plugin for measuring coverage; 2024. Pytest plugin integrating coverage measurement. https://pypi.org/project/pytest-cov/ [Accessed 9 October 2025].

[47] Garrett J. Scienceplots: matplotlib styles for scientific presentation; 2021. GitHub repository. https://github.com/garrettj403/SciencePlots [Accessed 9 October 2025].

[48] Müller F. Timezonefinder: Python library for fast and accurate timezone lookup; 2021. https://pypi.org/project/timezonefinder/[Accessed 9 October 2025].

[49] Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, Weckesser W, Bright J, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods 2020;17:261–72. https://doi.org/10.1038/s41592-019-0686-2

[50] Delattre P. Loguru: Python logging made (stupidly) simple. J Open Source Softw 2019;4(36):1209. https://doi.org/10.21105/joss.01209. https://joss.theoj.org/papers/10.21105/joss.01209.

[51] Chandra H, Sharma S, Lele PD, Rajaram G, Hanchinal A. Ionospheric measurements during the total solar eclipse of 11 August 1999. Earth Planets Space Jan 2007;59(1):59–64. https://doi.org/10.1186/BF03352023. ISSN 1880-5981.

[52] Barta V, et al. Multi-instrument analysis of medium-scale travelling disturbances during geomagnetic storms. Space Weather Space Climate 2025. https://www.swsc-journal.org/articles/swsc/full_html/2025/01/swsc240069/swsc240069.html.

[53] Uma G, Brahmanandam PS, Srinivasu VKD, Prasad DSVVD, Sai Gowtam V, Tulasi Ram S, Chu YH. Ionospheric responses to the 21 August 2017 Great American Solar Eclipse – a multi-instrument study. Adv Space Res 2020;65(1):74–85. https://doi.org/10.1016/j.asr.2019.09.010. ISSN 0273-1177. https://www.sciencedirect.com/science/article/pii/S0273117719306581.

[54] Zakharenkova I, et al. Multi-instrument observations of ionospheric super plasma bubbles over Europe. SWSC (Space Weather and Space Climate) 2025. https://www.swsc-journal.org/articles/swsc/full_html/2025/01/swsc240046/swsc240046.html.

[55] Chakraborty S, Qian L, Mrak S, Mabie J, Goncharenko L, Mclnerney JM, Bullett T. Formation of the ionospheric g-condition following the 2017 Great American eclipse. Earth Space Sci 2025;12(8):e2024EA004007. https://doi.org/10.1029/2024EA004007. https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2024EA004007.

[56] Barjatya A, Clayton R, Debchoudhury S, Zettergren M, Valentine H, Graves N, Conway R, Ribbens P, Milford J, Obenberger K, Holmes J, (Koki) Chau J, Lynch K, Mrak S, Bullett T. APEP: Eclipse sounding rocket campaign. In: European Geosciences Union General Assembly 2024 (egu24), EGU General Assembly Conference Abstracts, Apr 2024. https://doi.org/10.5194/egusphere-egu24-13772, p. 13772.

[57] Hosokawa K, Mayumi N, Yamamoto A, Saito S, Andoh S, Shinagawa H, Takahashi T, Sakai J, Tsuda T, Tomizawa I, Nakata H, Saito A, Shinohara M, Nishioka M, Tsugawa T, Ishii M. Visualizing the structure and motion of sporadic e layer using marine traffic radio signals. Earth Planets and Space May 2025;77(1):62. https://doi.org/10.1186/s40623-025-02193-x. ISSN 1880-5981.

[58] Clayton R, Barjatya A, Valentine H, Graves N, Milford J, Ribbens P, Conway R, Debchoudhury S, Zettergren MD, Bullett TW, Erickson PJ, Oreo AC. SpEED demon: a tech demo mission for simultaneous multipoint measurements of electrodynamics and neutral dynamics. In: AGU fall meeting abstracts, vol. 2022 of AGU Fall Meeting Abstracts, Dec 2022, SH32D–1794.